

Exercise 1

Part i

```
#include <iostream>
#include <vector>
#include <fstream>
#include <cmath>
#include "NR_C301/code/nr3.h"
#include "NR_C301/code/svd.h"

MatDoub loadMatrix(const std::string fileName) {
    std::ifstream fileToRead(fileName);

    // Error check
    if(!fileToRead.is_open()){
        std::cout << "Error during file loading..." << std::endl;
        exit(1);
    }

    int rows;
    int cols;

    fileToRead >> rows >> cols;

    MatDoub matrix(rows, cols);

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            fileToRead >> matrix[i][j];
        }
    }

    fileToRead.close();
    std::cout << "Successfully loaded file into matrix!" << std::endl;
    return matrix;
}

VecDoub loadVector(const std::string fileName) {
    std::ifstream fileToRead(fileName);

    // Error check
    if(!fileToRead.is_open()){
        std::cout << "Error during file loading..." << std::endl;
        exit(1);
    }

    int rows;
    int waste; // The .dat file still has a column variable, that is slow

    fileToRead >> rows;
    fileToRead >> waste;

    VecDoub vectorRes(rows);

    for (int i = 0; i < rows; ++i) {
        fileToRead >> vectorRes[i];
    }

    fileToRead.close();
    std::cout << "Successfully loaded file into vector!" << std::endl;
    return vectorRes;
}
```

```

int main(){
    // Load the .dat files in with the function. One was made for matrices and for vectors
    MatDoub fileA = loadMatrix("../Ex1A.dat");
    VecDoub fileB = loadVector("../Ex1b.dat");

    std::cout << "\nExercise 1.1" << std::endl;

    SVD fileAsvd(fileA);

    for (int i = 0; i < fileAsvd.w.size(); ++i) {
        std::cout << "W[" << i << "] = " << fileAsvd.w[i] << std::endl;
    }
}

```

```

pablo@pablodesktop:~/Docu
./run
Sucesfully loaded file!
Sucesfully loaded file!
Part 1
W[0] = 21.7792
W[1] = 14.1421
W[2] = 12.349
W[3] = 4.77724
W[4] = 4.24118
W[5] = 4.02939
W[6] = 3.21176
W[7] = 9.35182e-09
W[8] = 8.09617e-09
W[9] = 5.32927e-15

```

Part ii

Because we just calculated the w for the 10 values, I just used the smallest value, but bumped the decimal 1 point. This meant i used $5.32927e^{-14}$ as the threshold.

```

std::cout << "\nExercise 1.2" << std::endl;

VecDoub resVectorII(10);
double thresholdII = 5.32927e-14; // The lowest result is 5.32927e-15, so we are trying something here
double sumOfErrorII = 0.0;

fileAsvd.solve(fileB, resVectorII, thresholdII);

std::cout << "Solution with smallest W removed:" << std::endl;

for (int i = 0; i < resVectorII.size(); ++i) {
    std::cout << "x[" << i << "] = " << resVectorII[i] << std::endl;
}

// find the residual error
for (int i = 0; i < fileA.nrows(); ++i) {
    double Axi = 0.0; // Reset Axi

    for (int j = 0; j < fileA.ncols(); ++j) {
        Axi += fileA[i][j] * resVectorII[j];
    }

    double differenceII = Axi - fileB[i]; // Subtract the result of Axi with the result from the given data
    set

    sumOfErrorII += pow(differenceII, 2);
}

double residualErrorII = sqrt(sumOfErrorII);

std::cout << "The resiudal error ||Ax-b|| = " << residualErrorII << std::endl;

```

Solution with smallest W removed:

```
x[0] = -91.7636
x[1] = -32.6525
x[2] = -8705.59
x[3] = -15691.6
x[4] = 48.6542
x[5] = -94.9615
x[6] = 40115.6
x[7] = 32.1529
x[8] = 34487.1
x[9] = -47102.1
```

The residual error $\|Ax-b\| = 0.00403567$

pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build1\$

Part iii

I reused the code from before, but changed the threshold to $9.35182e^{-08}$. I could have refactored into a function, but this being an exam, time pressure was a factor...

```
std::cout << "\nExercise 1.3" << std::endl;

VecDoub resVectorIII(10);
double thresholdIII = 9.35182e-08; // The lowest result is 9.35182e-09, so we are trying something here, again
double sumOfErrorIII = 0.0;

fileAsvd.solve(fileB, resVectorIII, thresholdIII);

std::cout << "Solution with 3 smallest W's removed:" << std::endl;

for (int i = 0; i < resVectorIII.size(); ++i) {
    std::cout << "x[" << i << "] = " << resVectorIII[i] << std::endl;
}

// find the residual error
for (int i = 0; i < fileA.nrows(); ++i) {
    double Axi = 0.0; // Reset Axi

    for (int j = 0; j < fileA.ncols(); ++j) {
        Axi += fileA[i][j] * resVectorIII[j];
    }

    double differenceIII = Axi - fileB[i]; // Subtract the result of Axi with the result from the given data
    set

    sumOfErrorIII += pow(differenceIII, 2);
}

double residualErrorIII = sqrt(sumOfErrorIII);

std::cout << "The residual error  $\|Ax-b\| =$ " << residualErrorIII << std::endl;
```

Solution with smallest W removed:

```
x[0] = -91.7644
x[1] = -32.6507
x[2] = 14.5125
x[3] = 0.396323
x[4] = 48.6531
x[5] = -94.9615
x[6] = 3.1549
x[7] = 32.1541
x[8] = -35.3172
x[9] = -26.0314
```

The residual error $\|Ax-b\| = 0.00408194$

pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build1\$

Part iv

When we use a higher threshold, we exclude the smallest singular values, which forces the SVD calculation to produce a more stable and well-conditioned result. If we use a lower threshold (keeping the tiny values), we allow the algorithm the flexibility to fit the original data more perfectly, but this also allows microscopic noise or singularities to heavily impact the math. Therefore, by using the higher threshold, we accept a slightly higher residual error $\|Ax - b\|$, but the resulting solution vector x avoids massive, unstable spikes (like the ± 30.000 values) and remains physically meaningful.

Part v

Part V was implemented using a function, as i knew it was needed twice, unlike part 2 and 3.

```
void errorEstimate(SVD inputSVD, double threshold, int assignmentNum) {
    std::cout << "\nPart " << assignmentNum << std::endl;

    for (int i = 0; i < inputSVD.v.nrows(); ++i) {
        double sumTerm = 0.0;

        for (int j = 0; j < inputSVD.v.ncols(); ++j) {
            if (inputSVD.w[j] > threshold) {
                double term = inputSVD.v[i][j] / inputSVD.w[j];
                sumTerm += pow(term, 2);
            }
        }
        double errorXI = sqrt(sumTerm);

        std::cout << "Error on x[" << i << "] = +/- " << errorXI << std::endl;
    }
}
```

Implemented in main:

```
std::cout << "\nPart 5" << std::endl;

errorEstimate(fileAsvd, thresholdII, 2); // Part 2
errorEstimate(fileAsvd, thresholdIII, 3); // Part 3

return 0;
```

Part 5

Part 2

```
Error on x[0] = +/- 2.73068
Error on x[1] = +/- 6.23762
Error on x[2] = +/- 2.55087e+07
Error on x[3] = +/- 2.87554e+07
Error on x[4] = +/- 3.8268
Error on x[5] = +/- 0.268959
Error on x[6] = +/- 1.1734e+08
Error on x[7] = +/- 4.40274
Error on x[8] = +/- 6.32619e+07
Error on x[9] = +/- 8.62662e+07
```

Part 3

```
Error on x[0] = +/- 0.0736694
Error on x[1] = +/- 0.190284
Error on x[2] = +/- 0.0500246
Error on x[3] = +/- 0.0786107
Error on x[4] = +/- 0.228576
Error on x[5] = +/- 0.254833
Error on x[6] = +/- 0.0108749
Error on x[7] = +/- 0.210681
Error on x[8] = +/- 0.195425
Error on x[9] = +/- 0.146323
```

Because Part 2 only excluded the single smallest singular value (w), the second and third smallest values were still retained. Since both of these values are still near-zero, dividing by them inside the SVD solver causes extreme numerical instability, amplifying data noise, wildly inflating the components of the solution vector x , and drastically increasing the estimated error.

Part vi

```
std::cout << "\nExercise 1.6" << std::endl;

for (int j = 0; j < fileAsvd.v.ncols(); ++j) {
    if (fileAsvd.w[j] <= thresholdIII) {
        std::cout << "\nOrthonormal basis vector excluded w[" << j << "] = " << fileAsvd.w[j] << " : " <<
std::endl;

        for (int i = 0; i < fileAsvd.v.nrows(); ++i){
            std::cout << " v[" << i << "] = " << fileAsvd.v[i][j] << std::endl;
        }
    }
}
```

Part 6

```
Orthonormal basis vector excluded w[7] = 9.35182e-09 :
v[0] = 2.5484e-08
v[1] = -5.82198e-08
v[2] = 0.0993879
v[3] = -0.229423
v[4] = 3.56838e-08
v[5] = 1.83458e-10
v[6] = -0.457184
v[7] = -4.10748e-08
v[8] = 0.504731
v[9] = -0.68827

Orthonormal basis vector excluded w[8] = 8.09617e-09 :
v[0] = 1.29136e-09
v[1] = -2.7423e-09
v[2] = 0.187746
v[3] = 0.121451
v[4] = 1.45919e-09
v[5] = -6.78078e-10
v[6] = -0.86363
v[7] = -1.78417e-09
v[8] = -0.267192
v[9] = 0.364353

Orthonormal basis vector excluded w[9] = 5.32927e-15 :
v[0] = 0.30415
v[1] = -0.699544
v[2] = -9.02849e-09
v[3] = 1.87323e-08
v[4] = 0.425809
v[5] = -2.99422e-16
v[6] = 4.15311e-08
v[7] = -0.486639
v[8] = -4.1211e-08
```

By mathematical definition, the matrix V generated by Singular Value Decomposition is an orthogonal matrix, meaning its column vectors inherently form an orthonormal basis. The basis for the set of linear combinations excluded from the solution in part 3 consists of the specific column vectors in V corresponding to the singular values that were filtered out by the threshold.

Full code

```

#include <iostream>
#include <vector>
#include <fstream>
#include <cmath>
#include "NR_C301/code/nr3.h"
#include "NR_C301/code/svd.h"

MatDoub loadMatrix(const std::string fileName) {
    std::ifstream fileToRead(fileName);

    // Error check
    if(!fileToRead.is_open()){
        std::cout << "Error during file loading..." << std::endl;
        exit(1);
    }

    int rows;
    int cols;

    fileToRead >> rows >> cols;

    MatDoub matrix(rows, cols);

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            fileToRead >> matrix[i][j];
        }
    }

    fileToRead.close();
    std::cout << "Successfully loaded file into matrix!" << std::endl;
    return matrix;
}

VecDoub loadVector(const std::string fileName) {
    std::ifstream fileToRead(fileName);

    // Error check
    if(!fileToRead.is_open()){
        std::cout << "Error during file loading..." << std::endl;
        exit(1);
    }

    int rows;
    int waste; // The .dat file still has a column variable, that is slow

    fileToRead >> rows;
    fileToRead >> waste;

    VecDoub vectorRes(rows);

    for (int i = 0; i < rows; ++i) {
        fileToRead >> vectorRes[i];
    }

    fileToRead.close();
    std::cout << "Successfully loaded file into vector!" << std::endl;
    return vectorRes;
}

void errorEstimate(SVD inputSVD, double threshold, int assignmentNum) {
    std::cout << "\nPart " << assignmentNum << std::endl;

    for (int i = 0; i < inputSVD.v.nrows(); ++i) {
        double sumTerm = 0.0;

        for (int j = 0; j < inputSVD.v.ncols(); ++j) {
            if (inputSVD.w[j] > threshold) {
                double term = inputSVD.v[i][j] / inputSVD.w[j];
                sumTerm += pow(term, 2);
            }
        }
    }
}

```

```

    }
}
double errorXI = sqrt(sumTerm);

std::cout << "Error on x[" << i << "] = +/- " << errorXI << std::endl;
}
}

int main(){
// Load the .dat files in with the function. One was made for matrices and for vectors
MatDoub fileA = loadMatrix("../Ex1A.dat");
VecDoub fileB = loadVector("../Ex1b.dat");

std::cout << "\nExercise 1.1" << std::endl;

SVD fileAsvd(fileA);

for (int i = 0; i < fileAsvd.w.size(); ++i) {
    std::cout << "W[" << i << "] = " << fileAsvd.w[i] << std::endl;
}

//=====

std::cout << "\nExercise 1.2" << std::endl;

VecDoub resVectorII(10);
double thresholdII = 5.32927e-14; // The lowest result is 5.32927e-15, so we are trying something here
double sumOfErrorII = 0.0;

fileAsvd.solve(fileB, resVectorII, thresholdII);

std::cout << "Solution with smallest W removed:" << std::endl;

for (int i = 0; i < resVectorII.size(); ++i) {
    std::cout << "x[" << i << "] = " << resVectorII[i] << std::endl;
}

// find the residual error
for (int i = 0; i < fileA.nrows(); ++i) {
    double Axi = 0.0; // Reset Axi

    for (int j = 0; j < fileA.ncols(); ++j) {
        Axi += fileA[i][j] * resVectorII[j];
    }

    double differenceII = Axi - fileB[i]; // Subtract the result of Axi with the result from the given
data set

    sumOfErrorII += pow(differenceII, 2);
}

double residualErrorII = sqrt(sumOfErrorII);

std::cout << "The residual error ||Ax-b|| = " << residualErrorII << std::endl;

//=====

std::cout << "\nExercise 1.3" << std::endl;

VecDoub resVectorIII(10);
double thresholdIII = 9.35182e-08; // The lowest result is 9.35182e-09, so we are trying something here,
again
double sumOfErrorIII = 0.0;

fileAsvd.solve(fileB, resVectorIII, thresholdIII);

std::cout << "Solution with 3 smallest W's removed:" << std::endl;

for (int i = 0; i < resVectorIII.size(); ++i) {
    std::cout << "x[" << i << "] = " << resVectorIII[i] << std::endl;
}

```

```

// find the residual error
for (int i = 0; i < fileA.nrows(); ++i) {
    double Axi = 0.0; // Reset Axi

    for (int j = 0; j < fileA.ncols(); ++j) {
        Axi += fileA[i][j] * resVectorIII[j];
    }

    double differenceIII = Axi - fileB[i]; // Subtract the result of Axi with the result from the given
data set

    sumOfErrorIII += pow(differenceIII, 2);
}

double residualErrorIII = sqrt(sumOfErrorIII);

std::cout << "The resiudal error ||Ax-b|| = " << residualErrorIII << std::endl;

//=====================================================

std::cout << "\nExercise 1.5" << std::endl;

errorEstimate(fileAsvd, thresholdII, 2); // Part 2
errorEstimate(fileAsvd, thresholdIII, 3); // Part 3

//=====================================================

std::cout << "\nExercise 1.6" << std::endl;

for (int j = 0; j < fileAsvd.v.ncols(); ++j) {
    if (fileAsvd.w[j] <= thresholdIII) {
        std::cout << "\nOrthonormal basis vector exluded w[" << j << "] = " << fileAsvd.w[j] << " : " <<
std::endl;

        for (int i = 0; i < fileAsvd.v.nrows(); ++i){
            std::cout << " v[" << i << "] = " << fileAsvd.v[i][j] << std::endl;
        }
    }
}

return 0;
}

```

Exercise 2

Part i

The f function

```

double f(double x) {
    return (3.0*x*pow(cos(x),3) - 0.2*exp(pow(x,2)) + 0.7);
}

```

In main

```

int main() {

    std::cout << "Exercise 2.1\n" << std::endl;

    std::cout << std::fixed << std::setprecision(8);

    std::cout << "f(-1) = " << f(-1.0) << std::endl;
    std::cout << "f(1) = " << f(1.0) << std::endl;
}

```

```
    return 0;
}
```

Exercise 2.1

```
f(-1) = -0.31684218
f(1)  = 0.62952945
```

Part ii

This exercise was made with a modified version of the Ridders method from NR3, to show the actual iterations throughout, and stop it at the right point.

The Ridders function

```
template <class T>
Doub riddersMethod(T &func, const Doub x1, const Doub x2, const Doub xacc, int iterationCount) {
    // This is a modified version of the numerical recipes version
    Doub fl=func(x1);
    Doub fh=func(x2);
    if ((fl > 0.0 && fh < 0.0) || (fl < 0.0 && fh > 0.0)) {
        Doub xl=x1;
        Doub xh=x2;
        Doub ans=-9.99e99;
        for (Int j=0; j < iterationCount; j++) { // Set to only run for the number of iterations needed,
            instead of to max
                Doub xm=0.5*(xl+xh);
                Doub fm=func(xm);
                Doub s=sqrt(fm*fm-fl*fh);
                if (s == 0.0) return ans;
                Doub xnew=xm+(xm-xl)*((fl >= fh ? 1.0 : -1.0)*fm/s);
                if (abs(xnew-ans) <= xacc) return ans;
                ans=xnew;
                Doub fnew=func(ans);
                if (fnew == 0.0) return ans;
                if (SIGN(fm,fnew) != fm) {
                    xl=xm;
                    fl=fm;
                    xh=ans;
                    fh=fnew;
                }
                else if (SIGN(fl,fnew) != fl) {
                    xh=ans;
                    fh=fnew;
                }
                else if (SIGN(fh,fnew) != fh) {
                    xl=ans;
                    fl=fnew;
                }
                else throw("never get here.");
                if (abs(xh-xl) <= xacc) return ans;

                std::cout << "Iteration " << j+1 << " xi: " << xl << " yi: " << xh << std::endl;
            }
            return ans;
        }
    }
    else {
        if (fl == 0.0) return x1;
        if (fh == 0.0) return x2;
        throw("root must be bracketed in zriddr.");
    }
}
```

In main

```
std::cout << "\nExercise 2.2\n" << std::endl;
```

```
Doub res = riddersMethod(f, -1.0, 1.0, 0, 4); // Input is function, x1, x2, accuracy (set to 0 to not stop it running) and iteration count
```

```
std::cout << "Ridders solution after 4 iterations: " << res << std::endl;
```

Exercise 2.2

```
Iteration 1 xi: 0.00000000 yi: -0.74580238
Iteration 2 xi: -0.37290119 yi: -0.13347510
Iteration 3 xi: -0.25318815 yi: -0.17110160
Iteration 4 xi: -0.21214487 yi: -0.17218090
Ridders solution after 4 iterations: -0.17218090
```

Part iii

```
std::cout << "\nExercise 2.3\n" << std::endl;
```

```
res = riddersMethod(f, -1.0, 1.0, 0, 5);
```

```
std::cout << "Ridders solution after 5 iterations: " << res << std::endl;
```

Exercise 2.3

```
Iteration 1 xi: 0.00000000 yi: -0.74580238
Iteration 2 xi: -0.37290119 yi: -0.13347510
Iteration 3 xi: -0.25318815 yi: -0.17110160
Iteration 4 xi: -0.21214487 yi: -0.17218090
Iteration 5 xi: -0.19216288 yi: -0.17218678
Ridders solution after 5 iterations: -0.17218678
```

Estimating error for Ridders method, can be difficult, as noted in the slides from week 6. Round-off errors often set in before convergence constant C settles. However, we can try to estimate it based only on the data from iteration 4 and 5 in 2 ways. First, using the direct successive difference between y_4 and y_5 , we can see that the difference is slowing down significantly:

$$\begin{aligned} &|y_i - y_{i-1}| \\ &|(-0.17218678) - (-0.17218090)| = 0.00000588 \end{aligned}$$

Which is incredibly small. Beyond that we can also use a conservative value for C of 1000:

$$\begin{aligned} &C|d_k|^3 \\ &1000 * (0.00000588^3) \approx 2.03 * 10^{-13} \end{aligned}$$

Which again is quite accurate.

Part iv

Expected Order per f-computation:

According to the course summary table (slides week 6), the expected order of Ridders' method per iteration is 3. However, each iteration requires 2 new f computations (one at the midpoint z_i , and one at the new estimate x_{i+1}). Therefore, the expected order per f computation is:

$$3^{(1/2)} = \sqrt{3} \approx 1.732$$

Why Practical Convergence is Limited:

In practice, while the theoretical order is very high, floating-point limitations prevent the algorithm from perfectly achieving this mathematical ideal. As noted in the course slides (week 6), Ridders' method converges so rapidly that hardware round-off errors set in before the convergence constant C has time to mathematically settle. This numerical static is what ultimately limits the precision, rather than the theoretical efficiency of the algorithm itself.

Full code

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "NR_C301/code/nr3.h"

double f(double x) {
    return (3.0*x*pow(cos(x),3) - 0.2*exp(pow(x,2)) + 0.7);
}

template <class T>
Doub riddersMethod(T &func, const Doub x1, const Doub x2, const Doub xacc, int iterationCount) {
    // This is a modified version of the numerical recipes version
    Doub fl=func(x1);
    Doub fh=func(x2);
    if ((fl > 0.0 && fh < 0.0) || (fl < 0.0 && fh > 0.0)) {
        Doub xl=x1;
        Doub xh=x2;
        Doub ans=-9.99e99;
        for (Int j=0; j < iterationCount; j++) { // Set to only run for the number of iterations needed,
instead of to max
            Doub xm=0.5*(xl+xh);
            Doub fm=func(xm);
            Doub s=sqrt(fm*fm-fl*fh);
            if (s == 0.0) return ans;
            Doub xnew=xm+(xm-xl)*((fl >= fh ? 1.0 : -1.0)*fm/s);
            if (abs(xnew-ans) <= xacc) return ans;
            ans=xnew;
            Doub fnew=func(ans);
            if (fnew == 0.0) return ans;
            if (SIGN(fm,fnew) != fm) {
                xl=xm;
                fl=fm;
                xh=ans;
                fh=fnew;
            } else if (SIGN(fl,fnew) != fl) {
                xh=ans;
                fh=fnew;
            } else if (SIGN(fh,fnew) != fh) {
                xl=ans;
                fl=fnew;
            } else throw("never get here.");
            if (abs(xh-xl) <= xacc) return ans;

            std::cout << "Iteration " << j+1 << " xi: " << xl << " yi: " << xh << std::endl;
        }
        return ans;
    }
    else {
        if (fl == 0.0) return x1;
        if (fh == 0.0) return x2;
        throw("root must be bracketed in zriddr.");
    }
}

int main() {
    std::cout << "Exercise 2.1\n" << std::endl;

    std::cout << std::fixed << std::setprecision(8);

    std::cout << "f(-1) = " << f(-1.0) << std::endl;
    std::cout << "f(1) = " << f(1.0) << std::endl;

    //=====

    std::cout << "\nExercise 2.2\n" << std::endl;
```

```

    Doub res = riddersMethod(f, -1.0, 1.0, 0, 4); // Input is funtion, x1, x2, accuracy (set to 0 to not
stop it running) and iteration count
    std::cout << "Ridders solution after 4 iterations: " << res << std::endl;

//=====

    std::cout << "\nExercise 2.3\n" << std::endl;

    res = riddersMethod(f, -1.0, 1.0, 0, 5);
    std::cout << "Ridders solution after 5 iterations: " << res << std::endl;

    return 0;
}

```

Exercise 3

Part i

We need to separate and rewrite the initial 2nd order equation $\ddot{\theta}$, into 2 separate 1st order equations: $y_1(t) = \theta(t)$ is position $y_2(t) = \dot{\theta}(t)$ is velocity Then we write out the derivatives

$$\dot{y}_1(t) = y_2(t)$$

$$\dot{y}_2(t) = \left[-\frac{g}{L} + \frac{A\omega^2}{mL} \sin(\omega t) \right] \cos(y_1(t)) - k_D y_2(t) |y_2(t)|$$

The initial conditions at $t = 10$ are:

$$y_1(10) = 1.1$$

$$y_2(10) = -0.1$$

Part ii

```

#include <iostream>
#include <cmath>
#include <iomanip>

// Parameter init, needed in multiple functions, so they are global
const double m = 1.0; // Mass of the pendulum
const double L = 0.5; // Length of the rod
const double g = 9.81; // Gravity
const double kD = 0.2;
const double omega = 100.0;
const double A = 0.05;

double getAccel(double t, double theta, double thetaDot) {
    double sec1 = -g / L;
    double sec2 = (A * pow(omega, 2)) / (m * L) * sin(omega * t);
    double sec3 = cos(theta);
    double sec4 = kD * thetaDot * std::abs(thetaDot);

    return ((sec1 + sec2) * sec3 - sec4);
}

int main() {

    // Start conditions
    double t = 10.0;
    double theta = 1.1; // y1
    double thetaDot = -0.1; // y2

    std::cout << std::fixed << std::setprecision(10);
    std::cout << "thetaDoubleDot(10) = " << getAccel(t, theta, thetaDot) << std::endl;

    return 0;
}

```

```
pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build3$ ./run
thetaDoubleDot(10) = 366.1717965691
pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build3$
```

Part iii

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <fstream>

// Parameter init, needed in multiple functions, so they are global
const double m = 1.0;           // Mass of the pendulum
const double L = 0.5;           // Length of the rod
const double g = 9.81;           // Gravity
const double kD = 0.2;
const double omega = 100.0;
const double A = 0.05;

double getAccel(double t, double theta, double thetaDot) {
    double sec1 = -g / L;
    double sec2 = (A * pow(omega, 2)) / (m * L) * sin(omega * t);
    double sec3 = cos(theta);
    double sec4 = kD * thetaDot * std::abs(thetaDot);

    return ((sec1 + sec2) * sec3 - sec4);
}

double ODEmidpoint(double N, double tStart, double tEnd, double theta, double thetaDot) {
    double tInternal = tStart;
    double thetaInternal = theta;
    double thetaDotInternal = thetaDot;
    double h = (tEnd - tStart) / N;

    std::ofstream graphData;
    graphData.open("graphingData.csv");
    graphData << "t, theta\n";

    for (int i = 0; i < N; ++i) {
        if (i % 10 == 0) {
            graphData << tInternal << "," << thetaInternal << "\n";
        }

        double k1Theta = thetaDotInternal;
        double k1ThetaDot = getAccel(tInternal, thetaInternal, thetaDotInternal);

        double tMid = tInternal + 0.5 * h;
        double thetaMid = thetaInternal + 0.5 * h * k1Theta;
        double thetaDotMid = thetaDotInternal + 0.5 * h * k1ThetaDot;

        double k2Theta = thetaDotMid;
        double k2ThetaDot = getAccel(tMid, thetaMid, thetaDotMid);

        thetaInternal += h * k2Theta;
        thetaDotInternal += h * k2ThetaDot;
        tInternal += h;
    }
    graphData << tInternal << "," << thetaInternal << "\n";
    graphData.close();

    return thetaInternal;
}

int main() {
    // Start conditions
    double t = 10.0;
    double theta = 1.1;           // y1
    double thetaDot = -0.1;       // y2
```

```

std::cout << "\nPart 3\n" << std::endl;

std::cout << "theta(20) = " << ODEmidpoint(100000, t, 20, theta, thetaDot) << std::endl;

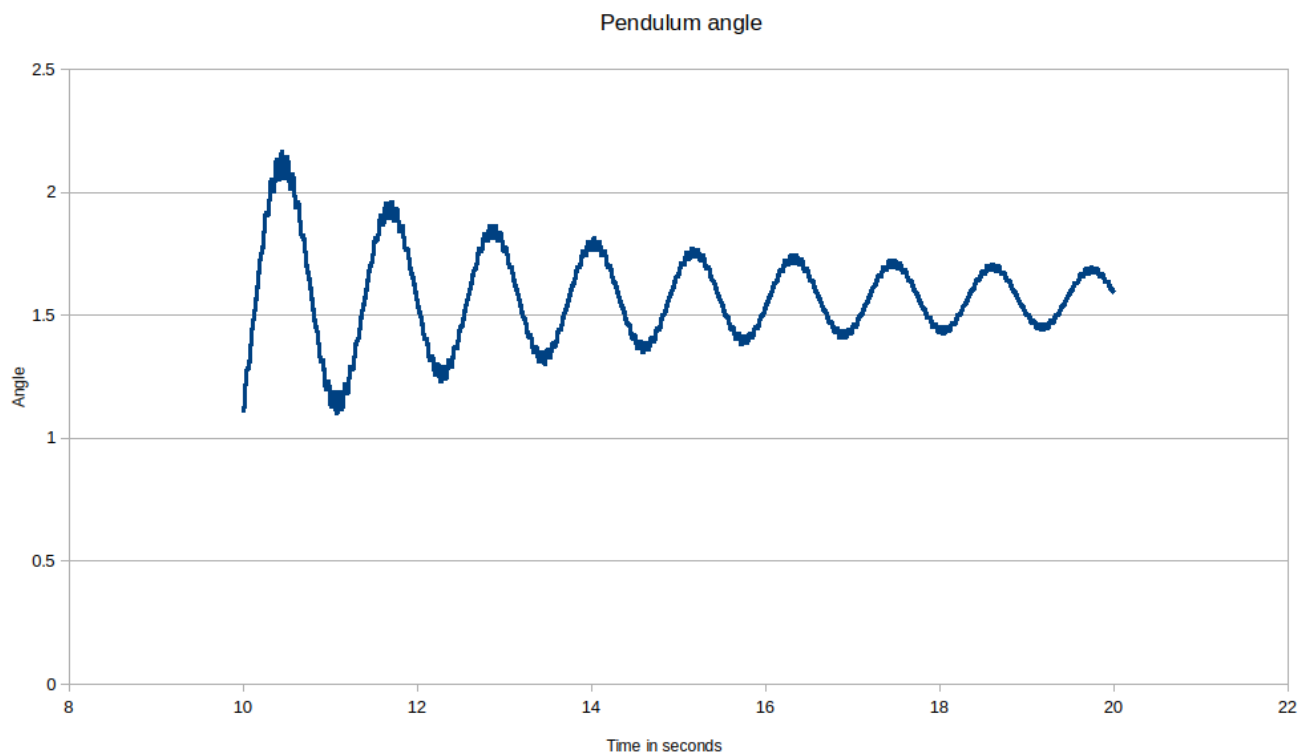
return 0;
}

```

Part 3

```
theta(20) = 1.5946364458
```

```
pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build3$
```



To make the graph I exported every 1 millisecond, or every 10th iteration. Lower resolution could do it, but with a CSV file it was easy to get this level of detail. Going further up in resolution, would make the data less legible, though it was possible by just changing the hard-coded limit.

Part iv

```

void printTable(int runNumber, int N, double res, double accuracy) {
    std::cout << "|" << std::setw(3) << runNumber << " |" << std::setw(14) << N << " |" << std::setw(14)
<< std::setprecision(10) << res << " |" ;
    if (accuracy != MAXFLOAT) {
        std::cout << std::setw(14) << std::setprecision(8) << accuracy << " |" << std::endl;
    } else {
        std::cout << std::setw(14) << "" << " |" << std::endl;
    }
}

int main() {
    // Start conditions
    double t = 10.0;
    double theta = 1.1; // y1
    double thetaDot = -0.1; // y2

    std::cout << "\nPart 4\n" << std::endl;

    std::cout << "|" << std::setw(3) << "Try" << " |" << std::setw(14) << "N" << " |" << std::setw(14) <<
"Result" << " |" << std::setw(14) << "Accuracy" << " |" << std::endl;
}

```

```

int NtoTest = 1000;
double minAccuracy = 1e-5;
std::vector<double> results;

for (int k = 1; k <= 25; ++k) {
    results.push_back(ODEmidpoint(NtoTest, t, 20, theta, thetaDot));

    double accuracy = MAXFLOAT;

    if (results.size() >= 2) {
        accuracy = std::abs((results[1] - results[0]) / 3.0);
        results.erase(results.begin());
    }

    printTable(k, NtoTest, results[0], accuracy);

    if (accuracy <= minAccuracy) {
        std::cout << "\nAccuracy threshold reached at N = " << NtoTest << std::endl;
        break;
    }
    NtoTest *= 2;
}
}

```

This code is still using the rest of the functions made in the previous part, like the ODEmidpoint, but are not included here for simplicity's

Part 4

Try	N	Result	Accuracy
1	1000	1.7056490688	
2	2000	1.5980083816	0.03588023
3	4000	1.4882323769	0.03659200
4	8000	1.5656720479	0.02581322
5	16000	1.5875558512	0.00729460
6	32000	1.5930090901	0.00181775
7	64000	1.5943689664	0.00045329
8	128000	1.5947087480	0.00011326
9	256000	1.5947936894	0.00002831
10	512000	1.5948149255	0.00000708

sake. Accuracy threshold reached at N = 512000

Explanation of Accuracy Estimate: The problem states to assume a global order of convergence $p = 2$ for the Midpoint method. To estimate the accuracy of our numerical solution without knowing the exact analytical solution, we utilize Richardson error estimation. By comparing the results from a coarse grid ($N/2$ subdivisions) and a fine grid (N subdivisions), the estimated absolute error is calculated as:

$$\text{Estimated Error} \approx \frac{|\theta_N(20) - \theta_{N/2}(20)|}{2^p - 1} = \frac{|\theta_N(20) - \theta_{N/2}(20)|}{3}$$

By sequentially doubling the number of subdivisions starting from $N = 1000$, we track this error estimate.

Required Subdivisions: Based on the iterative calculation (see table above), the estimated error drops below the target threshold of 10^{-5} at $N = 512000$. At this subdivision, we have reached the required accuracy.

Full code

```

#include <iostream>
#include <cmath>
#include <iomanip>
#include <fstream>
#include <vector>

// Parameter init, needed in multiple functions, so they are global
const double m = 1.0;           // Mass of the pendulum
const double L = 0.5;           // Length of the rod

```

```

const double g = 9.81;           // Gravity
const double kD = 0.2;
const double omega = 100.0;
const double A = 0.05;

double getAccel(double t, double theta, double thetaDot) {
    double sec1 = -g / L;
    double sec2 = (A * pow(omega, 2)) / (m * L) * sin(omega * t);
    double sec3 = cos(theta);
    double sec4 = kD * thetaDot * std::abs(thetaDot);

    return ((sec1 + sec2) * sec3 - sec4);
}

double ODEmidpoint(double N, double tStart, double tEnd, double theta, double thetaDot) {
    double tInternal = tStart;
    double thetaInternal = theta;
    double thetaDotInternal = thetaDot;
    double h = (tEnd - tStart) / N;

    std::ofstream graphData;
    graphData.open("graphingData.csv");
    graphData << "t, theta\n";

    for (int i = 0; i < N; ++i) {
        if (i % 10 == 0) {
            graphData << tInternal << "," << thetaInternal << "\n";
        }

        double k1Theta      = thetaDotInternal;
        double k1ThetaDot    = getAccel(tInternal, thetaInternal, thetaDotInternal);

        double tMid          = tInternal + 0.5 * h;
        double thetaMid       = thetaInternal + 0.5 * h * k1Theta;
        double thetaDotMid    = thetaDotInternal + 0.5 * h * k1ThetaDot;

        double k2Theta      = thetaDotMid;
        double k2ThetaDot    = getAccel(tMid, thetaMid, thetaDotMid);

        thetaInternal += h * k2Theta;
        thetaDotInternal += h * k2ThetaDot;
        tInternal += h;
    }
    graphData << tInternal << "," << thetaInternal << "\n";
    graphData.close();

    return thetaInternal;
}

void printTable(int runNumber, int N, double res, double accuracy) {
    std::cout << " | " << std::setw(3) << runNumber << " | " << std::setw(14) << N << " | " << std::setw(14)
    << std::setprecision(10) << res << " | " ;
    if (accuracy != MAXFLOAT) {
        std::cout << std::setw(14) << std::setprecision(8) << accuracy << " | " << std::endl;
    } else {
        std::cout << std::setw(14) << "" << " | " << std::endl;
    }
}

int main() {
    // Start conditions
    double t = 10.0;
    double theta = 1.1;           // y1
    double thetaDot = -0.1;       // y2

    //=====

    std::cout << "Part 2\n" << std::endl;

    std::cout << std::fixed << std::setprecision(10);
    std::cout << "thetaDoubleDot(10) = " << getAccel(t, theta, thetaDot) << std::endl;

```

```
//=====

std::cout << "\nPart 3\n" << std::endl;

std::cout << "theta(20) = " << ODEmidpoint(100000, t, 20, theta, thetaDot) << std::endl;

//=====

std::cout << "\nPart 4\n" << std::endl;

std::cout << " | " << std::setw(3) << "Try" << " | " << std::setw(14) << "N" << " | " << std::setw(14) <<
"Result" << " | " << std::setw(14) << "Accuracy" << " | " << std::endl;

int NtoTest = 1000;
double minAccuracy = 1e-5;
std::vector<double> results;

for (int k = 1; k <= 25; ++k) {
    results.push_back(ODEmidpoint(NtoTest, t, 20, theta, thetaDot));

    double accuracy = MAXFLOAT;

    if (results.size() >= 2) {
        accuracy = std::abs((results[1] - results[0]) / 3.0);
        results.erase(results.begin());
    }

    printTable(k, NtoTest, results[0], accuracy);

    if (accuracy <= minAccuracy) {
        std::cout << "\nAccuracy threshold reached at N = " << NtoTest << std::endl;
        break;
    }
    NtoTest *= 2;
}
return 0;
}
```

Exercise 4

Part i

```
#include <iostream>
#include <cmath>
#include <vector>
#include <iomanip>

double f(double x) {
    double inner = sin( -(x*x) + 5.0*x - 4.0);
    if (inner < 0.0) {
        inner = 0.0;    // This safety is necessary as because sqrt of 0 is impossible without complex
        // numbers, resulting in nasty errors
    }

    return exp(2.0*x) * sqrt(inner);
}

double simpsonMethod(double a, double b, int n) {
    double h = (b-a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; ++i) {
        double x = a + i * h;
        double weight;
        if (i % 2 == 0) {
            weight = 2.0;
        }
    }
}
```

```

        else {
            weight = 4.0;
        }
        sum += weight * f(x);
    }
    return (h/3.0) * sum;
}

int main() {
    std::cout << "Exercise 2.1\n" << std::endl;

    double a = 1.0;
    double b = 4.0;

    std::cout << std::fixed << std::setprecision(10);
    std::cout << " | " << std::setw(2) << "k" << " | " << std::setw(10) << "N" << " | " << std::setw(15) <<
    "Integral Result" << std::endl;

    for (int k = 1; k <= 20; ++k) {
        int n = pow(2, k); // Number of intervals
        double result = simpsonMethod(a, b, n);
        std::cout << " | " << std::setw(2) << k << " | " << std::setw(10) << n << " | " << std::setw(15) <<
        result << std::endl;
    }

    return 0;
}

```

Exercise 4.1

k	N	Integral Result
1	2	261.8259166477
2	4	761.3346878378
3	8	1034.4168650655
4	16	1113.9247450382
5	32	1138.4472375335
6	64	1146.6338943855
7	128	1149.4557720288
8	256	1150.4416427697
9	512	1150.7881963945
10	1024	1150.9103745297
11	2048	1150.9535103022
12	4096	1150.9687504224
13	8192	1150.9741367358
14	16384	1150.9760407528
15	32768	1150.9767138657
16	65536	1150.9769518367
17	131072	1150.9770359703
18	262144	1150.9770657157
19	524288	1150.9770762322
20	1048576	1150.9770799504

Part ii

I completely updated the code in this second part, so the entire code is added, even though there are parts that repeat from part i

```

#include <iostream>
#include <cmath>
#include <vector>
#include <iomanip>

double f(double x) {
    double inner = sin( -(x*x) + 5.0*x - 4.0);
    if (inner < 0.0) {
        inner = 0.0; // This safety is necessary as because sqrt of 0 is impossible without complex
        numbers, resulting in nasty errors
    }
}

```

```

    return exp(2.0*x) * sqrt(inner);
}

double simpsonMethod(double a, double b, int n) {
    double h = (b-a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; ++i) {
        double x = a + i * h;
        double weight;
        if (i % 2 == 0) {
            weight = 2.0;
        }
        else {
            weight = 4.0;
        }
        sum += weight * f(x);
    }
    return (h/3.0) * sum;
}

double richardsonExtrapolation (double ah1, double ah2, double alphak) {
    return (alphak * ah2 - ah1) / (alphak - 1.0);
}

int main() {
    std::cout << "Exercise 4.2\n" << std::endl;

    double a = 1.0;
    double b = 4.0;
    double alpha = 2.0;
    std::vector<double> results;

    std::cout << std::fixed << std::setprecision(10);
    std::cout << " | " << std::setw(2) << "k" << " | " << std::setw(10) << "N" << " | " << std::setw(15) <<
    "Integral Result" << " | " << std::setw(15) << "Richardson" << " | " << std::setw(15) << "Error" << " | " <<
    std::setw(12) << "alpha^k" << " | " << std::endl;

    for (int k = 1; k <= 20; ++k) {
        int n = pow(2, k); // Number of intervals

        results.push_back(simpsonMethod(a, b, n));

        std::cout << " | " << std::setw(2) << k << " | " << std::setw(10) << n << " | " << std::setw(15) <<
        results[k-1];

        if (k>2) { // When 3 iterations are done, richardson extrapolation can be done:
            double kRich = log2(std::abs((results[k-2] - results[k-3]) / (results[k-1] - results[k-2])));
            // the result is going 1 further back than according to the formula, because k starts at 1, but the vector is
            // 0 indexed
            double alphaK = pow(alpha, kRich);
            double richRes = richardsonExtrapolation(results[k-2], results[k-1], alphaK);
            double errEst = std::abs(results[k-1]-richRes);

            std::cout << " | " << std::setw(15) << richRes << " | " << std::setw(15) << errEst << " | " <<
            std::setw(12) << alphaK << " | " << std::endl;
        }
        else { // Without 3 iterations, Richardson cannot be done, N/A is finishing the table
            std::cout << " | " << std::setw(15) << "N/A" << " | " << std::setw(15) << "N/A" << " | " <<
            std::setw(12) << "N/A" << " | " << std::endl;
        }
    }

    return 0;
}

```

Exercise 4.2

k	N	Integral Result	Richardson	Error	alpha^k
1	2	261.8259166477	N/A	N/A	N/A
2	4	761.3346878378	N/A	N/A	N/A
3	8	1034.4168650655	1363.7680875274	329.3512224620	1.8291518555
4	16	1113.9247450382	1146.5814720284	32.6567269902	3.4346554998
5	32	1138.4472375335	1149.3838275217	10.9365899882	3.2422430138
6	64	1146.6338943855	1150.7366138225	4.1027194369	2.9954220555
7	128	1149.4557720288	1150.9400816572	1.4843096284	2.9011381381
8	256	1150.4416427697	1150.9710204734	0.5293777036	2.8623201056
9	512	1150.7881963945	1150.9760521734	0.1878557789	2.8447855413
10	1024	1150.9103745297	1150.9769036178	0.0665290881	2.8364618943
11	2048	1150.9535103022	1150.9770507787	0.0235404764	2.8324086458
12	4096	1150.9687504224	1150.9770764964	0.0083260739	2.8304089450
13	8192	1150.9741367358	1150.9770810164	0.0029442806	2.8294158001
14	16384	1150.9760407528	1150.9770818131	0.0010410603	2.8289209068
15	32768	1150.9767138657	1150.9770819537	0.0003680880	2.8286739042
16	65536	1150.9769518367	1150.9770819786	0.0001301419	2.8285503344
17	131072	1150.9770359703	1150.9770819830	0.0000460126	2.8284888817
18	262144	1150.9770657157	1150.9770819837	0.0000162680	2.8284580406
19	524288	1150.9770762322	1150.9770819838	0.0000057516	2.8284509477
20	1048576	1150.9770799504	1150.9770819839	0.0000020335	2.8284180255

pablo@pablodesktop:~/Documents/Uni/Numerical-methods-exercises/Workspace/build3\$

Part iii

Estimated vs. Expected Order The expected theoretical order of convergence for Simpson's Method is $k = 4$ (according to the slides from week 8). However, the data reveals a different empirical order. Based on the ratio of successive errors (α^k), the table shows the value stabilizing at approximately 2.8284 for the final iterations.

Since the step size ratio is $\alpha = 2$, we can calculate the exact estimated order k :

$$2^k \approx 2.8284 \implies k = \log_2(2.8284) \approx 1.5$$

The estimated order of convergence is approximately 1.5, which is significantly lower than the expected order of 4.

Explanation for the Difference: Simpson's Method requires the function to be completely "smooth", meaning its slope must be measurable and finite everywhere.

Our function involves a square root: $\sqrt{\sin(-x^2 + 5x - 4)}$. At the exact edges of our integration ($x = 1$ and $x = 4$), the value inside the square root becomes zero. Mathematically, the derivative of a square root function at zero is vertical, or infinity.

Because the slope drops completely vertical at the boundaries, the function is not "smooth." This sudden vertical drop breaks the mathematical rules required for Simpson's 4th-order accuracy, causing its efficiency to drop to the observed order of ≈ 1.5 .

Part iv

Computing the Accuracy Estimate To verify the accuracy of the integration at $N = 1,048,576$ ($k = 20$), we use Richardson error estimation. As defined in the course material, the difference between the base Simpson's approximation and the extrapolated value provides a reliable estimate of the truncation error.

The estimated absolute error is calculated using the difference between the Simpson's result $A(h)$ and the Richardson refined result A_R :

$$\text{Estimated Error} \approx |A(h) - A_R|$$

Estimated Accuracy at $N = 2^{20}$ By analyzing the generated data (can be seen in part 2) at our final subdivision $k = 20$ ($N = 1,048,576$), the estimated absolute error is $\approx 2.03 \times 10^{-6}$.

Full code

The full and final code was uploaded during Part ii.

